# 03. CONGEST: волны, расстояния и минимальный остов

**Что вы узнаете.** Как строить дерево и обнаруживать завершение без глобального таймера; когда агрегирование экономит связь; почему доставка всех данных требует конвейера; как устроены точные расстояния и фазы Борувки. Это заменяет прежние лекции 06–07 и часть 01.

## 1. Волна и echo

В этой лекции граф связен, у вершин уникальные ID из $[n^c]$, вес ребра, если он есть, известен обоим концам и кодируется $O(\log n)$ битами. CONGEST передаёт по каждому направлению ребра не меньше $B$ и не больше константы $B$ бит, где $B = \Theta(\log n)$; когда в одном сообщении нужны несколько ID и вес, константу в $B$ предполагаем достаточной. Корень $r$ заранее известен всем либо уже выбран отдельно.

**Алгоритм 3.1 (BFS-волна).** Корень в первом раунде посылает `DISCOVER`. Неоткрытая вершина, впервые получившая его в раунде $t$, записывает уровень $d(v) = t$ и выбирает родителя среди отправителей этого раунда по минимальному ID. В следующем раунде она передаёт `DISCOVER` дальше. Ответы `REJECT` на поздние открытия и `ACK` после завершения детей позволяют обнаружить конец работы.

**Теорема 3.2.** Уровни алгоритма 3.1 равны расстояниям $d(v) = \mathrm{dist}_G(r, v)$; родительские рёбра образуют BFS-дерево. При echo-подтверждениях корень узнаёт о завершении через $O(D)$ раундов, где $D = \max_v d(v)$, без предварительного знания $D$.

**Доказательство.** Сообщение за раунд проходит не более одного ребра, поэтому раньше расстояния $d_G(r, v)$ прийти не может. По кратчайшему пути оно приходит не позже этого раунда; индукция по длине пути даёт равенство. У каждого некорневого узла родитель на предыдущем уровне; уровни строго убывают вдоль родительских рёбер к $r$, поэтому циклов нет, а все вершины соединены с корнем. Для echo каждый узел ждёт ответ на каждую посланную попытку открытия и подтверждения детей. Лист подтверждает сразу после ответов соседей; индукция по высоте поддерева показывает, что подтверждение корню дойдёт после волны не более чем за $O(D)$ дополнительных раундов. Корень не объявит конец, пока есть неотвеченная ветвь.

**Пример.** В треугольнике два соседа корня откроются одновременно. Ребро между ними не войдёт в BFS-дерево; оба процесса всё равно обязаны ответить друг другу на попытку открытия, иначе один может ждать вечно. Волны и BFS в CONGEST: Hirvonen–Suomela, гл. 5.

## 2. Агрегирование и конвейер

**Теорема 3.3 (convergecast).** Пусть на вершинах дано $x(v)$ и определена ассоциативная операция $\oplus$ с нейтральным элементом. Если каждое промежуточное значение $\bigoplus_{u \in S} x(u)$ помещается в одно CONGEST-сообщение, корень BFS-дерева вычисляет $\bigoplus_{v \in V} x(v)$ за $O(D)$ раундов и $n - 1$ сообщений вверх по дереву.

**Доказательство.** Лист передаёт $x(v)$. Внутренняя вершина после получения значений детей передаёт $s(v) = x(v) \oplus \bigoplus_{u \text{ ребёнок}} s(u)$. Индукция по поддеревьям показывает, что $s(v)$ есть агрегат ровно его вершин. Высота дерева не превышает $D$,

поэтому значение корня готово через не более $D$ шагов после начала восходящей фазы. Каждое некорневое ребро несёт ровно одно значение вверх.

При $x(v) = 1$ и обычном сложении корень получает $n$ в $O(D)$ раундов, поскольку счётчик до $n$ занимает $O(\log n)$ бит. При $x(v) = \deg(v)$ он получает $2|E|$. Для XOR $B$-битных слов агрегат всегда имеет длину $B$. Но если требуется восстановить *каждую* из $n$ строк, XOR недостаточен: нужны пакеты.

**Лемма 3.4 (разрезная информационная оценка).** Пусть через единственное ребро разреза левая часть должна передать правой одну из $2^K$ независимых строк, а сообщения слева направо имеют не более $B$ бит. Любой детерминированный протокол точного восстановления требует $T \geq \lceil K/B \rceil$ раундов.

**Доказательство.** За $T$ раундов возможны не более $2^{BT}$ транскриптов на этом направлении ребра. Если двум различным строкам соответствует один транскрипт, правая часть с одинаковыми собственными входами и одинаковыми полученными сообщениями выдаст одинаковый ответ, ошибившись на одной из строк. Поэтому транскриптов не меньше $2^K$ и $BT \geq K$.

**Утверждение 3.5 (конвейер на пути).** Если источник должен отправить $K$ бит по пути длины $\ell$ с пакетами по $B$ бит, то после разбиения на $p = \lceil K/B \rceil$ пакетов первый приходит через $\ell$ раундов, каждый следующий — на один раунд позже. Все пакеты получены за $\ell + p - 1$ раундов.

**Доказательство.** Источник посылает пакет $j$ в раунде $j$, а каждая промежуточная вершина пересылает его в следующем раунде после получения. Индукция по позиции ребра даёт момент прибытия $j$-го пакета $j + \ell - 1$. Пакеты разных номеров занимают разные рёбра одновременно, поэтому конфликтов нет.

Это пример различия **distance** и **congestion**: нижняя оценка расстояния — $\ell$, пропускной способности — $p$, но их простое сложение с поправкой $-1$ возникает из необходимости последнему пакету пройти весь путь. На звезде $k$ листьев передают центру $k$ пакетов за один раунд, поскольку используют разные рёбра. Подробнее о полосе: Hirvonen–Suomela, §5.1–5.3.

## 3. Агрегат, доставка всех данных и остановка

Дерево BFS часто называют «инфраструктурой» алгоритма, но одна и та же инфраструктура даёт разные оценки в зависимости от объёма результата. Пусть у каждого листа корневого дерева есть $B$-битное слово. Если корню нужен только XOR, каждый внутренний процесс хранит один $B$-битный накопитель. После получения слов от детей он берёт XOR с собственным словом и посылает один пакет родителю. Пакет каждого ребра используется ровно один раз; глубина дерева даёт $O(D)$ раундов. Для подсчёта числа вершин суммарная величина не превышает $n$ и помещается в $O(\log n)$ бит. Для восстановления всех слов результат содержит $nB$ бит, а узкое ребро у корня может пропустить лишь $B$ бит за раунд. Если все источники лежат по другую сторону такого ребра, требуется $\Omega(n)$ раундов даже при диаметре 3.

**Утверждение 3.5a (двусторонняя цена разреза).** Если по разные стороны одного моста лежат независимые строки длины $K_L$ и $K_R$ бит и каждая сторона должна восстановить другую, а каждое направление моста передаёт $B$ бит за раунд, то при доступности строк на концах моста точное минимальное число раундов равно $\max\{\lceil K_L/B \rceil, \lceil K_R/B \rceil\}$.

**Доказательство.** При фиксированном правом входе правый процесс должен различать $2^{K_L}$ левых строк, значит, транскрипт слева направо содержит не меньше $K_L$ бит. Аналогично обратное направление требует $K_R$ бит. Это даёт максимум двух нижних оценок. В каждом раунде оба конца одновременно отправляют следующий блок своей строки в своём направлении, поэтому максимум также достижим.

Для реальной сети к этой цене добавляется доставка строк к концам моста и от них к получателям. Нельзя складывать две информационные оценки, поскольку два направления физического ребра работают одновременно. Наоборот, когда *один* получатель должен собрать много пакетов, нагрузки разных источников суммируются на общем направлении узкого ребра. Это различие полезно и в задаче 09.4.

Остановка echo зависит от того, что узел подтвердит завершение только после подтверждений всех детей. Если пакет данных в процессе ещё идёт вверх, подтверждение нельзя отправлять раньше него, иначе корень объявит завершение до прихода последнего результата. Для XOR можно объединить подтверждение и итоговый агрегат в одном сообщении; для потока всех слов нужен отдельный маркер конца потока после последнего пакета. Доказательство остановки должно учитывать порядок доставки, а не только конечное значение.

## 4. Кратчайшие пути

**Определение 3.6 (SSSP).** В графе с неотрицательными целыми весами $w(e) \in [0, n^c]$ и источником $s$ каждый процесс $v$ должен вывести $\mathrm{dist}_w(s, v)$, минимум суммы весов по путям от $s$ до $v$.

**Алгоритм 3.7 (синхронный Bellman–Ford).** Положим $d_0(s) = 0$, $d_0(v) = \infty$ при $v \neq s$; затем для $t \geq 0$ каждый процесс вычисляет $d_{t+1}(v) = \min\{d_t(v), \min_{u \sim v}(d_t(u) + w(uv))\}$. В раунде передаётся одно текущее значение расстояния каждому соседу.

**Теорема 3.8.** После $t$ раундов $d_t(v)$ равен минимальному весу пути из $s$ в $v$ с не более чем $t$ рёбрами. После $n - 1$ раундов получены точные расстояния в CONGEST.

**Доказательство.** При $t = 0$ верен только путь длины ноль в источнике. На шаге $t+1$ любой допустимый путь либо уже имеет не более $t$ рёбер, либо заканчивается ребром $uv$ после пути из $s$ в $u$ с не более $t$ рёбрами. Рекурсия перебирает ровно эти варианты. Неотрицательные веса позволяют выбрать простой кратчайший путь: удаление цикла не увеличивает вес. Простой путь содержит не более $n - 1$ рёбер. Значения не превышают $(n - 1)n^c$, поэтому кодируются $O(\log n)$ битами; специальный символ $\infty$ тоже конечен по коду.

Этот алгоритм не обещает $O(D)$ на взвешенном графе: оптимальный путь может иметь $n - 1$ рёбер даже при малом невзвешенном диаметре. Для невзвешенного графа BFS-волна использует геометрию и завершается за $O(D)$. Оценка времени Bellman–Ford относится к числу *рёбер* в путях, не к сумме весов.

## 5. Фазы Борувки для MST

**Определение 3.9 (MST и разрез).** Остовное дерево $T$ связного взвешенного графа имеет вес $w(T) = \sum_{e \in T} w(e)$. MST минимизирует этот вес. Для $S \subsetneq V$ исходящее ребро имеет один конец в $S$. При ничьих сравниваем пары

$(w(e), \mathrm{ID}(u), \mathrm{ID}(v))$ в согласованном порядке, получая строгий полный порядок рёбер.

**Лемма 3.10 (свойство разреза).** При строгом порядке минимальное исходящее ребро $e$ любого разреза содержится в единственном минимальном остове по этому порядку.

**Доказательство.** Пусть $T$ — минимальное дерево без $e$. Добавление $e$ создаёт цикл, содержащий другое ребро $f$ того же разреза. По минимальности $e$ имеем $e < f$ в строгом порядке. Замена $f$ на $e$ даёт дерево с меньшим упорядоченным набором рёбер, противореча минимальности. Лексикографическое разрешение равных исходных весов выбирает один из MST исходных весов.

**Алгоритм 3.11 (Борувка).** Начинаем с леса из отдельных вершин. В каждой фазе каждая его компонента выбирает минимальное исходящее ребро по общему строгому порядку; все выбранные рёбра добавляются одновременно, компоненты сливаются.

**Теорема 3.12.** Алгоритм 3.11 заканчивается не более чем за $\lceil \log_2 n \rceil$ фаз и выдаёт MST. Выбранные рёбра каждой фазы не образуют цикла.

**Доказательство.** По лемме 3.10 каждый выбранный край входит в один и тот же уникальный MST для строгого порядка, поэтому циклов среди них нет и полученный лес остаётся частью MST. Если перед фазой было $k > 1$ компонент, каждая выбирает ребро к другой; каждая новая компонента содержит не менее двух старых, значит, их число не больше $k/2$. После $\lceil \log_2 n \rceil$ фаз остаётся одна компонента, а лес, включённый в MST, уже есть весь MST.

Доказательство числа фаз ещё не равно быстрой распределённой реализации. Для поиска минимального исходящего ребра процессы должны знать метки компонент соседей, агрегировать предложения внутри каждой компоненты и разослать результат. Малые компоненты можно обслуживать внутри них; большие создают нагрузку на сеть. Ghaffari, §2.2.1–2.2.2 начинает построение более быстрого CONGEST-алгоритма именно с этого разделения. Здесь доказаны комбинаторная корректность и число фаз, а не полный результат о почти оптимальном числе раундов.

**Проверка понимания.** Для каждого алгоритма укажите отдельно раунды, максимальный размер сообщения, общую связь и критерий остановки. Утверждение «$O(D)$ раундов» может быть правильным для XOR и ложным для восстановления всех строк на том же дереве.